PYTHON PROGRAMMING • ACTIVITY-BASED LEARNING

10 CASE STUDY SOLUTIONS

A classroom-ready project and practical record

THINK	DESIGN	CODE	TEST

WHAT'S INSIDE

Problem statement · Objectives · Assumptions · Algorithm · Flowchart · Pseudocode · Python code · Test cases · Output preview · Conclusion

Selected case studies: 12, 13, 17, 27, 28, 29, 31, 35, 42 and 53.

Prepared from the uploaded Python Activity-Based Learning case-study sheet. Numeric thresholds and sample data are illustrative where the sheet does not specify them.

Presentation tip: use the problem statement and algorithm to explain the idea, then demonstrate the Python program and its output.

CONTENTS

Ten practical programs using beginner-level Python concepts

NO.	CASE STUDY	MAIN CONCEPTS
01	12. Attendance Recovery Computation	Loops, percentage, conditions
02	13. ATM Transaction Sequence Simulation	Lists, tuples, loops, validation
03	17. Student Performance Trend Analysis	Nested lists, averages, comparisons
04	27. Advanced Numeric Pattern Generator	Nested loops, pattern generation
05	28. Repeated Transaction Detection	Lists, tuple unpacking, duplicate checks
06	29. Password Policy Validation Engine	Strings, character checks, conditions
07	31. String-Based Examination Answer Evaluation	Strings, lists, counters, scoring
08	35. Text Compression Using Character Runs	Strings, indexing, counters
09	42. Student Marks Matrix Analysis	Nested lists, row/column calculations
10	53. Food Waste Monitoring System	Lists, percentages, threshold checks

HOW TO USE THIS BOOKLET

Each program uses fixed sample data so that the output is repeatable. To test another scenario, edit the values in the program and run it again in Python 3. The flowcharts are written as clear arrow sequences that can be redrawn as standard flowchart symbols if your faculty specifically requires diagram shapes.

CASE STUDY 01

Attendance Recovery Computation

ASSIGNMENT CASE NO. 12	LEVEL Beginner

01 PROBLEM STATEMENT

Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

Problem interpretation

A student needs to know whether their attendance meets the required percentage. If it does not, calculate how many consecutive classes they must attend, assuming they attend every upcoming class.

02 OBJECTIVES

- Calculate attendance percentage from classes attended and conducted.
- Check whether the eligibility threshold has been met.
- Find the minimum future classes needed to reach the threshold.

03 ASSUMPTIONS

- Attendance is represented by whole numbers.
- The student attends every future class being calculated.
- The required attendance threshold is 75%.
- Classes conducted must be greater than zero and attended classes cannot exceed conducted classes.

04 ALGORITHM

1. Start.
2. Set classes attended, classes conducted, and the required percentage.
3. Validate the attendance values.
4. Calculate current percentage = attended / conducted × 100.
5. If the percentage meets the threshold, report eligibility.
6. Otherwise, increase the future-class count one by one until the percentage meets the threshold.
7. Display the percentage, eligibility, and minimum classes needed.
8. Stop.

05 FLOWCHART (TEXT FLOW)

START → Read attendance data → Validate values → Calculate percentage → Eligible already? → Yes: Display eligible → No: Count future attended classes until threshold is reached → Display result → END

06 PSEUDOCODE

```
BEGIN
  attended ← 42; conducted ← 60; threshold ← 75
  IF conducted <= 0 OR attended < 0 OR attended > conducted THEN
    PRINT invalid data
  ELSE
    percentage ← attended / conducted * 100
    future ← 0
    WHILE (attended + future) / (conducted + future) * 100 < threshold
      future ← future + 1
    END WHILE
    PRINT percentage, future
  END IF
END
```

07 PYTHON SOURCE CODE

```
attended = 42
conducted = 60
threshold = 75

if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance data")
else:
    percentage = attended / conducted * 100
    print(f"Current attendance: {percentage:.2f}%")

    if percentage >= threshold:
        print("Status: Eligible")
        print("Additional classes needed: 0")
    else:
        future = 0
        while (attended + future) / (conducted + future) * 100 < threshold:
            future += 1

        print("Status: Not eligible yet")
        print("Additional consecutive classes to attend:", future)
        final_percentage = (attended + future) / (conducted + future) * 100
        print(f"Attendance after those classes: {final_percentage:.2f}%")
```

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
42 attended / 60 conducted	70.00%	12 classes needed
45 attended / 60 conducted	75.00%	Already eligible
61 attended / 60 conducted	Invalid	Invalid attendance data

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

ATTENDANCE RECOVERY

```
Current attendance: 70.00%
Status: Not eligible yet
Additional consecutive classes to attend: 12
Attendance after those classes: 75.00%
```

10 CONCLUSION

The program calculates the current attendance percentage and uses a loop to find the smallest number of consecutive classes needed to reach the eligibility threshold.

CASE STUDY 02

ATM Transaction Sequence Simulation

ASSIGNMENT CASE NO. 13	LEVEL Beginner

01 PROBLEM STATEMENT

Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

Problem interpretation

A simple ATM simulation must update an account balance for deposits and withdrawals, reject invalid transactions, and prevent withdrawals above the available balance or a per-transaction limit.

02 OBJECTIVES

- Process a list of transactions.
- Update the balance correctly after each accepted transaction.
- Reject invalid amounts and withdrawals that break the rules.
- Summarize accepted and rejected transactions.

03 ASSUMPTIONS

- The opening balance is non-negative.
- Amounts must be positive.
- The maximum withdrawal per transaction is ₹10,000.
- A withdrawal cannot exceed the available balance.

04 ALGORITHM

1. Start with an opening balance and a list of transactions.
2. Set accepted and rejected counters to zero.
3. For each transaction, validate the amount.
4. For a deposit, add the amount to the balance.
5. For a withdrawal, check the limit and available balance before subtracting.
6. Reject unknown transaction types.
7. Display each transaction result and the final summary.
8. Stop.

05 FLOWCHART (TEXT FLOW)

START → Set opening balance and transactions → Process next transaction → Valid amount/type? → Apply deposit or check withdrawal rules → Update or reject → More transactions? → Yes: repeat → No: print summary → END

06 PSEUDOCODE

```
BEGIN
  balance ← 5000
  accepted ← 0; rejected ← 0
  FOR each transaction (type, amount)
    IF amount <= 0 THEN reject
    ELSE IF type = deposit THEN balance ← balance + amount
    ELSE IF type = withdrawal AND amount <= 10000 AND amount <= balance THEN
      balance ← balance - amount; accept
    ELSE reject
    END IF
  END FOR
  PRINT balance, accepted, rejected
END
```

07 PYTHON SOURCE CODE

```
balance = 5000
transactions = [
    ("deposit", 2000),
    ("withdrawal", 1500),
    ("withdrawal", 7000),
    ("deposit", -200),
    ("transfer", 500)
]
withdrawal_limit = 10000
accepted = 0
rejected = 0

for transaction_type, amount in transactions:
    print(f"Transaction: {transaction_type} ₹{amount}")
    if amount <= 0:
        print("Result: Rejected - amount must be positive")
        rejected += 1
    elif transaction_type == "deposit":
        balance += amount
        print("Result: Accepted")
        accepted += 1
    elif transaction_type == "withdrawal":
        if amount > withdrawal_limit:
            print("Result: Rejected - withdrawal limit exceeded")
            rejected += 1
        elif amount > balance:
            print("Result: Rejected - insufficient balance")
            rejected += 1
        else:
            balance -= amount
            print("Result: Accepted")
            accepted += 1
    else:
        print("Result: Rejected - unknown transaction type")
        rejected += 1

print("\n--- Summary ---")
print("Final balance: ₹", balance)
print("Accepted transactions:", accepted)
print("Rejected transactions:", rejected)
```

08 TEST CASES

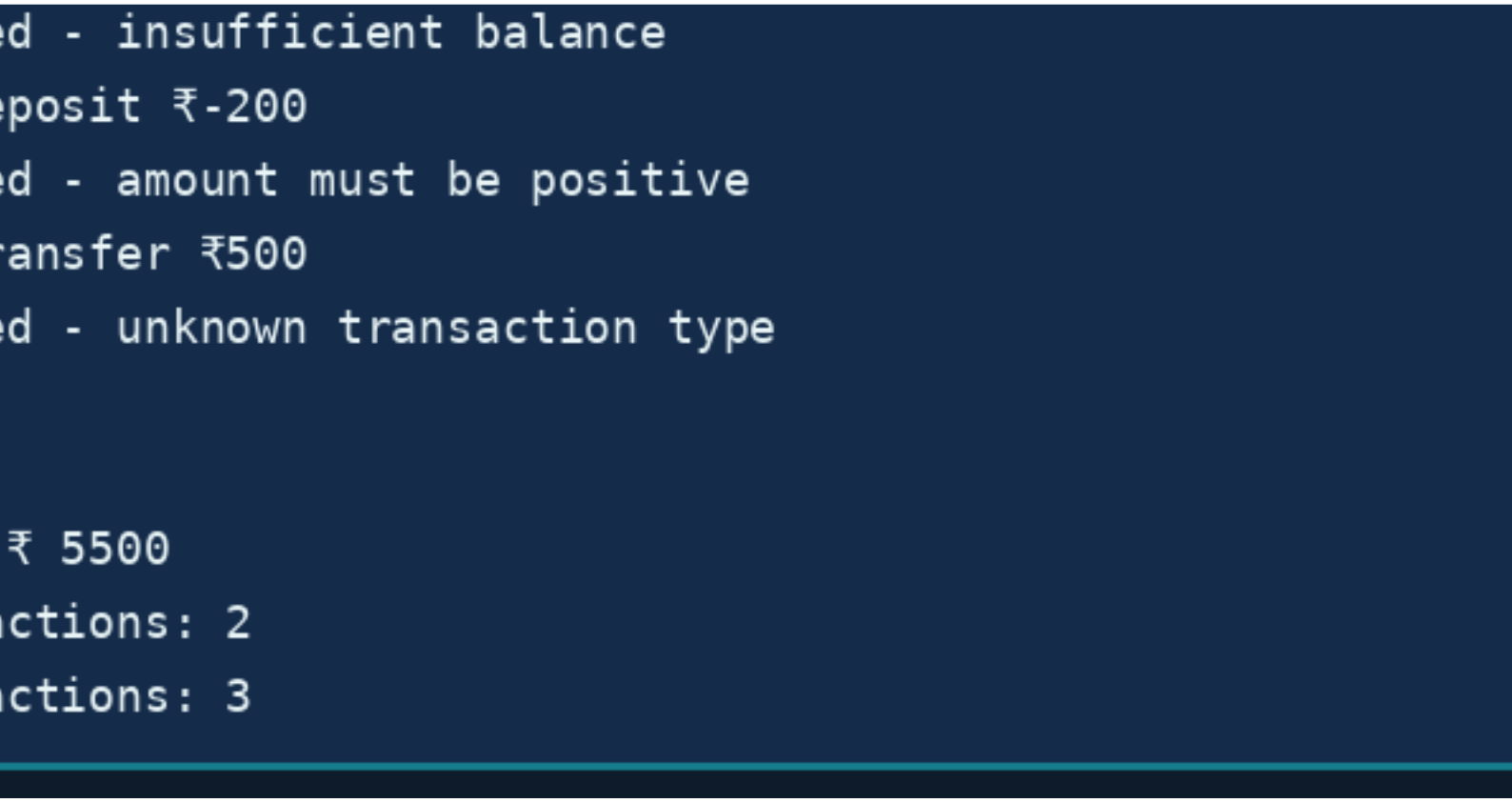
INPUT / SCENARIO	CHECK	EXPECTED RESULT
deposit ₹2000 from ₹5000	Balance becomes ₹7000	Accepted
withdrawal ₹7000 from ₹5500	Balance unchanged	Rejected: insufficient balance
deposit ₹-200	Balance unchanged	Rejected: invalid amount

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

ATM TRANSACTION SUMMARY

```
Transaction: deposit ₹2000
Result: Accepted
Transaction: withdrawal ₹1500
Result: Accepted
Transaction: withdrawal ₹7000
Result: Rejected - insufficient balance
Transaction: deposit ₹-200
Result: Rejected - amount must be positive
Transaction: transfer ₹500
Result: Rejected - unknown transaction type
--- Summary ---
Final balance: ₹ 5500
Accepted transactions: 2
Rejected transactions: 3
```



10 CONCLUSION

The simulation demonstrates lists, tuples, loops, conditional statements, and validation while maintaining a correct running balance.

CASE STUDY 03

Student Performance Trend Analysis

ASSIGNMENT CASE NO. 17	LEVEL Beginner

01 PROBLEM STATEMENT

Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

Problem interpretation

A student’s marks across three assessments are stored subject-wise. The program compares the first and last assessment, identifies improvement or decline, and flags subjects with a low final score.

02 OBJECTIVES

- Store marks in lists.
- Calculate change between assessments.
- Identify subjects below a defined mark threshold.
- Summarize overall performance.

03 ASSUMPTIONS

- Marks are out of 100.
- Each subject has marks for three assessments.
- A final mark below 40 is considered weak.
- The sample data contains valid marks from 0 to 100.

04 ALGORITHM

1. Start.
2. Store subject names and three assessment scores in lists.
3. For each subject, calculate change from Assessment 1 to Assessment 3.
4. Classify the trend as improved, declined, or unchanged.
5. Flag final marks below 40.
6. Calculate the average final mark.
7. Display the subject-wise analysis and overall average.
8. Stop.

05 FLOWCHART (TEXT FLOW)

START → Read subject marks → For each subject calculate change → Classify trend → Check final mark below 40 → Record weak subject if needed → Calculate average → Display report → END

06 PSEUDOCODE

```
BEGIN
  subjects ← [Maths, English, Python]
  marks ← [[55, 60, 72], [70, 68, 65], [35, 38, 39]]
  total ← 0
  FOR each subject index i
    change ← marks[i][2] - marks[i][0]
    classify change
    IF marks[i][2] < 40 THEN mark subject weak
    total ← total + marks[i][2]
  END FOR
  PRINT total / number_of_subjects
END
```

07 PYTHON SOURCE CODE

```
subjects = ["Maths", "English", "Python"]
marks = [
    [55, 60, 72],
    [70, 68, 65],
    [35, 38, 39]
]
weak_limit = 40
final_total = 0
weak_subjects = []

print("STUDENT PERFORMANCE TREND")
for i in range(len(subjects)):
    first_mark = marks[i][0]
    final_mark = marks[i][2]
    change = final_mark - first_mark

    if change > 0:
        trend = "Improved"
    elif change < 0:
        trend = "Declined"
    else:
        trend = "Unchanged"

    if final_mark < weak_limit:
        weak_subjects.append(subjects[i])

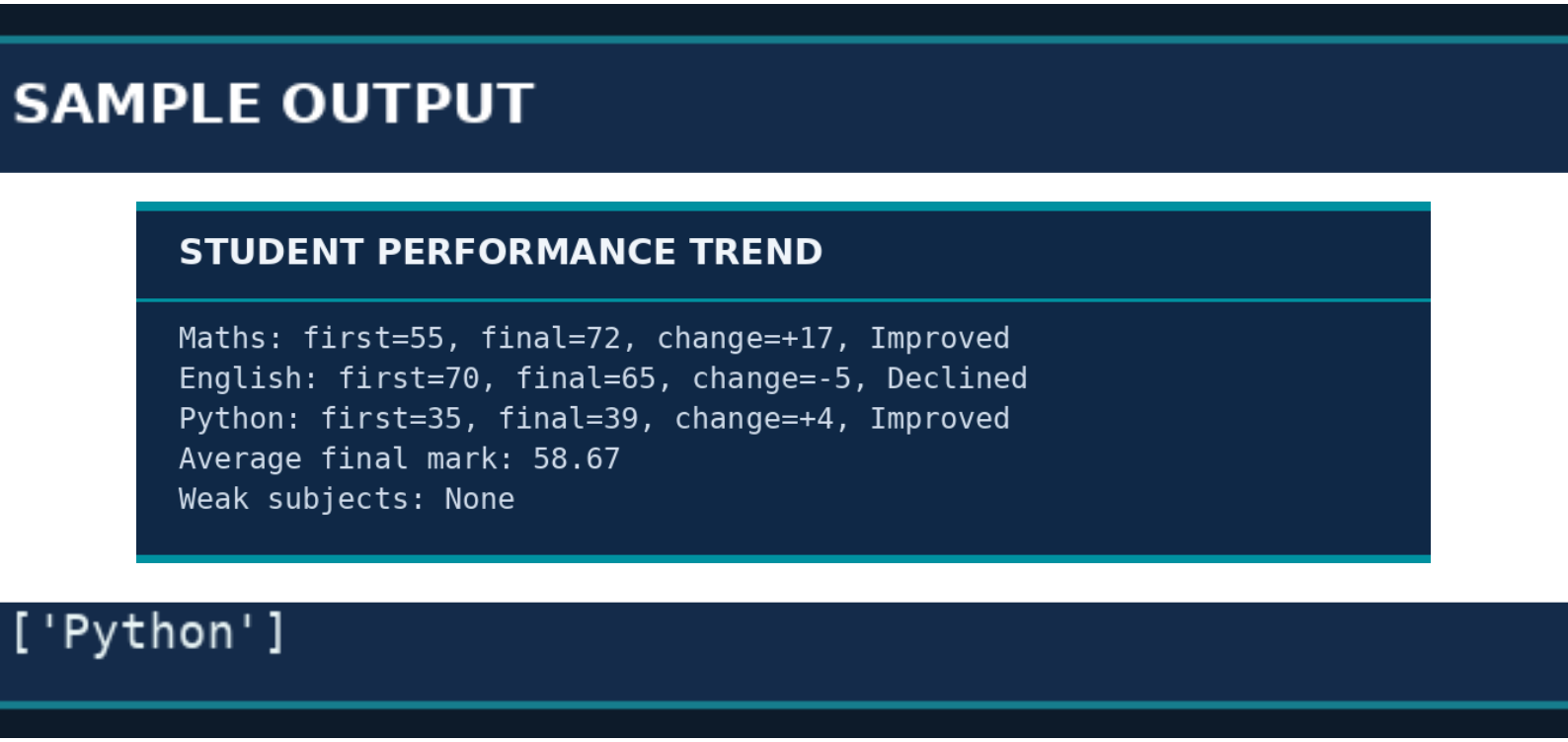
    final_total += final_mark
    print(f"{subjects[i]}: first={first_mark}, final={final_mark}, change={change:+}, {trend}")

average = final_total / len(subjects)
print(f"Average final mark: {average:.2f}")
print("Weak subjects:", weak_subjects if weak_subjects else "None")
```

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
Maths: 55 → 72	Change +17	Improved
English: 70 → 65	Change -5	Declined
Python final mark 39	Below 40	Flagged weak

09 SAMPLE OUTPUT PREVIEW



10 CONCLUSION

The program uses nested lists to compare assessments, calculate changes, identify weak subjects, and report the average final mark.

CASE STUDY 04

Advanced Numeric Pattern Generator

ASSIGNMENT CASE NO. 27	LEVEL Beginner

01 PROBLEM STATEMENT

Produce a hollow diamond or symmetric numerical pattern based on user-specified dimensions, with validation for invalid dimensions and carefully aligned output.

Problem interpretation

Generate a symmetric hollow diamond using a positive odd-free size value. The program validates the size and prints stars only on the boundary of the diamond.

02 OBJECTIVES

- Use nested loops to generate a pattern.
- Validate the requested size.
- Print a symmetric hollow diamond.
- Understand how row and column positions control output.

03 ASSUMPTIONS

- The size is a positive integer.
- The example uses size 5 for the upper and lower halves, making a 9-row diamond.
- A star is printed at the left and right boundary; interior positions are spaces.

04 ALGORITHM

1. Start.
2. Set size to 5.
3. If size is not positive, print an error.
4. Print the upper half including its middle row using nested loops.
5. Print the lower half using nested loops.
6. Stop.

05 FLOWCHART (TEXT FLOW)

START → Set size → Is size positive? → No: show error → Yes: loop through upper rows → loop through lower rows → print hollow diamond → END

06 PSEUDOCODE

```
BEGIN
  size ← 5
  IF size <= 0 THEN PRINT error
  ELSE
    FOR row from 1 to size
      print leading spaces
      FOR column from 1 to 2*row-1
        print * at boundary, otherwise space
      END FOR
      print newline
    END FOR
    FOR row from size-1 down to 1
      repeat boundary printing
    END FOR
  END IF
END
```

07 PYTHON SOURCE CODE

```
size = 5

if size <= 0:
    print("Size must be a positive integer")
else:
    # Upper half, including the widest row
    for row in range(1, size + 1):
        print(" " * (size - row), end="")
        for column in range(1, 2 * row):
            if column == 1 or column == 2 * row - 1:
                print("*", end="")
            else:
                print(" ", end="")
        print()

    # Lower half
    for row in range(size - 1, 0, -1):
        print(" " * (size - row), end="")
        for column in range(1, 2 * row):
            if column == 1 or column == 2 * row - 1:
                print("*", end="")
            else:
                print(" ", end="")
        print()
```

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
size = 5	9-row hollow diamond	Valid
size = 1	One star	Valid
size = 0	No pattern	Error message

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

HOLLOW DIAMOND



10 CONCLUSION

The pattern generator demonstrates nested loops, conditions, string repetition, and alignment through spaces.

CASE STUDY 05

Repeated Transaction Detection

ASSIGNMENT CASE NO. 28	LEVEL Beginner

01 PROBLEM STATEMENT

Process transaction identifiers and amounts in a list, detect duplicate transactions, identify threshold violations, and generate a list of suspicious entries.

Problem interpretation

A transaction log may contain duplicate identifiers or unusually large amounts. This program finds duplicate IDs and flags amounts above a specified threshold.

02 OBJECTIVES

- Process transaction records stored as tuples.
- Detect duplicate transaction IDs.
- Flag transactions above a threshold.
- Build and display a suspicious-entry list.

03 ASSUMPTIONS

- Each transaction is a tuple of (ID, amount).
- IDs should be unique.
- An amount above ₹5,000 is flagged for review.
- A flagged transaction is suspicious for review, not proof of fraud.

04 ALGORITHM

1. Start.
2. Store transactions as tuples in a list.
3. Create an empty list for seen IDs and suspicious records.
4. For each transaction, check whether its ID has appeared before.
5. Also check whether its amount exceeds the threshold.
6. Add any record meeting either condition to the suspicious list.
7. Display the suspicious records.
8. Stop.

05 FLOWCHART (TEXT FLOW)

START → Read tuple records → For each record check duplicate ID and amount threshold → Add flagged record → More records? → repeat → Display suspicious records → END

06 PSEUDOCODE

```
BEGIN
  transactions ← [(T101, 1200), (T102, 6500), (T101, 1200), (T104, 800)]
  seen ← []; suspicious ← []
  FOR each (id, amount)
    duplicate ← id in seen
    IF duplicate OR amount > 5000 THEN add record to suspicious
    IF id not in seen THEN add id to seen
  END FOR
  PRINT suspicious
END
```

07 PYTHON SOURCE CODE

```
transactions = [
    ("T101", 1200),
    ("T102", 6500),
    ("T101", 1200),
    ("T104", 800),
    ("T105", 7200)
]
threshold = 5000
seen_ids = []
suspicious = []

for transaction_id, amount in transactions:
    reasons = []

    if transaction_id in seen_ids:
        reasons.append("duplicate ID")
    else:
        seen_ids.append(transaction_id)

    if amount > threshold:
        reasons.append("amount above threshold")

    if reasons:
        suspicious.append((transaction_id, amount, reasons))

print("SUSPICIOUS TRANSACTIONS FOR REVIEW")
if suspicious:
    for transaction_id, amount, reasons in suspicious:
        print(f"{transaction_id} | ₹{amount} | {', '.join(reasons)}")
else:
    print("None")
```

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
T101 appears twice	Duplicate ID	Flagged
Amount ₹6500	Above ₹5000	Flagged
T104, ₹800	Unique and below threshold	Not flagged

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

SUSPICIOUS TRANSACTIONS

```
T102 | ₹6500 | amount above threshold
T101 | ₹1200 | duplicate ID
T105 | ₹7200 | amount above threshold
```

10 CONCLUSION

The program applies list membership checks and tuple unpacking to find duplicate identifiers and high-value transactions for review.

CASE STUDY 06

Password Policy Validation Engine

ASSIGNMENT CASE NO. 29	LEVEL Beginner

01 PROBLEM STATEMENT

Evaluate sample password strings against minimum length, character-category, repetition, and prohibited-substring rules; explain each failed condition.

Problem interpretation

Check sample passwords against a basic policy and report every rule that a password fails. This is a classroom validation exercise, not a complete security audit.

02 OBJECTIVES

- Check minimum password length.
- Check uppercase, lowercase, digit, and special-character requirements.
- Check for a repeated character sequence and a prohibited substring.
- Report all failed rules rather than stopping at the first failure.

03 ASSUMPTIONS

- Minimum length is 8 characters.
- The password must include uppercase, lowercase, a digit, and a special character.
- The substring 'password' is prohibited, ignoring letter case.
- Three consecutive identical characters are not allowed.

04 ALGORITHM

1. Start.
2. Store sample passwords.
3. For each password, create an empty list of failures.
4. Check length and each required character category.
5. Check the prohibited substring and three repeated consecutive characters.
6. If no failures exist, label the password as passing the sample policy.
7. Otherwise print every failure.
8. Stop.

05 FLOWCHART (TEXT FLOW)

START → Get sample password → Check all policy rules → Record every failed rule → Failures empty? → Yes: Pass → No: Print failures → Next password → END

06 PSEUDOCODE

```
BEGIN
  passwords ← [Hello@123, passwordA1!, abc111XY!]
  FOR each password
    failures ← []
    check length, uppercase, lowercase, digit, special character
    check prohibited substring
    check 3 consecutive identical characters
    IF failures empty THEN PRINT pass ELSE PRINT failures
  END FOR
END
```

07 PYTHON SOURCE CODE

```
passwords = ["Hello@123", "passwordA1!", "abc111XY!", "short"]
prohibited = "password"

for password in passwords:
    failures = []

    if len(password) < 8:
        failures.append("must contain at least 8 characters")
    if not any(ch.isupper() for ch in password):
        failures.append("needs an uppercase letter")
    if not any(ch.islower() for ch in password):
        failures.append("needs a lowercase letter")
    if not any(ch.isdigit() for ch in password):
        failures.append("needs a digit")
    if not any(not ch.isalnum() for ch in password):
        failures.append("needs a special character")
    if prohibited in password.lower():
        failures.append("contains prohibited word 'password'")

    repeated = False
    for i in range(len(password) - 2):
        if password[i] == password[i + 1] == password[i + 2]:
            repeated = True
    if repeated:
        failures.append("has 3 identical consecutive characters")

    print(f"Password: {password}")
    if failures:
        print("Status: FAIL")
        for failure in failures:
            print("-", failure)
    else:
        print("Status: PASS")
    print()
```

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
Hello@123	All rules satisfied	PASS
passwordA1!	Contains prohibited word; no uppercase	FAIL
abc111XY!	Three repeated consecutive digits	FAIL

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

PASSWORD POLICY CHECK

```
Password: Hello@123
Status: PASS

Password: passwordA1!
Status: FAIL
- contains prohibited word 'password'

Password: abc111XY!
Status: FAIL
- has 3 identical consecutive characters

Password: short
Status: FAIL
- must contain at least 8 characters
- needs an uppercase letter
- needs a digit
- needs a special character
```

```
ercase letter
ibited word 'password'

1XY!

al consecutive characters

at least 8 characters
ercase letter
al character
```

10 CONCLUSION

The program uses string methods, loops, conditions, and a list of error messages to explain which password rules are not met.

CASE STUDY 07

String-Based Examination Answer Evaluation

ASSIGNMENT CASE NO. 31	LEVEL Beginner

01 PROBLEM STATEMENT

Compare answer strings against an answer key, calculate correct, incorrect, and unanswered responses, apply negative marking, and produce student-wise results.

Problem interpretation

Evaluate a student's multiple-choice responses against an answer key. Blank answers are unanswered, wrong answers receive negative marks, and the final score is calculated using the supplied policy.

02 OBJECTIVES

- Compare each answer with an answer key.
- Count correct, incorrect, and unanswered questions.
- Apply negative marking to incorrect responses.
- Calculate and display the final score.

03 ASSUMPTIONS

- Each correct answer earns 4 marks.
- Each incorrect answer loses 1 mark.
- An unanswered question earns 0 marks.
- The answer and key lists have the same length.

04 ALGORITHM

1. Start.
2. Store the answer key and student's responses.
3. Initialize correct, incorrect, and unanswered counters.
4. Compare each response with the corresponding key entry.
5. Update the appropriate counter.
6. Calculate score = correct × 4 − incorrect × 1.
7. Display counts and score.
8. Stop.

05 FLOWCHART (TEXT FLOW)

START → Store key and responses → Compare each answer → Correct / incorrect / blank? → Update counters → Calculate score with negative marking → Print result → END

06 PSEUDOCODE

```
BEGIN
  key ← [A, C, B, D, A]
  answers ← [A, B, blank, D, C]
  correct ← 0; incorrect ← 0; unanswered ← 0
  FOR each position i
    IF answer blank THEN unanswered++
    ELSE IF answer = key[i] THEN correct++
    ELSE incorrect++
  END FOR
  score ← correct * 4 - incorrect
  PRINT counters and score
END
```

07 PYTHON SOURCE CODE

```
answer_key = ["A", "C", "B", "D", "A"]
student_answers = ["A", "B", "", "D", "C"]
marks_per_correct = 4
penalty_per_wrong = 1

if len(answer_key) != len(student_answers):
    print("Error: answer list and key must have equal length")
else:
    correct = 0
    incorrect = 0
    unanswered = 0

    for i in range(len(answer_key)):
        answer = student_answers[i].strip().upper()
        if answer == "":
            unanswered += 1
        elif answer == answer_key[i]:
            correct += 1
        else:
            incorrect += 1

    score = correct * marks_per_correct - incorrect * penalty_per_wrong
    print("EXAMINATION RESULT")
    print("Correct answers:", correct)
    print("Incorrect answers:", incorrect)
    print("Unanswered:", unanswered)
    print("Final score:", score, "out of", len(answer_key) * marks_per_correct)
```

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
A matches A	Correct count increases	4 marks
B differs from C	Incorrect count increases	−1 mark
Blank response	Unanswered count increases	0 marks

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

EXAMINATION RESULT

```
Correct answers: 2
Incorrect answers: 2
Unanswered: 1
Final score: 6 out of 20
```

10 CONCLUSION

The evaluator demonstrates string comparison, list indexing, loops, counters, and a simple negative-marking formula.

CASE STUDY 08

Text Compression Using Character Runs

ASSIGNMENT CASE NO. 35	LEVEL Beginner

01 PROBLEM STATEMENT

Encode a string by grouping consecutive repeated characters and compare original and encoded lengths, including cases where encoding is not beneficial.

Problem interpretation

Run-length encoding replaces each consecutive group of the same character with the character followed by its count. If the encoded version is not shorter, keep the original string.

02 OBJECTIVES

- Identify consecutive repeated characters.
- Create a run-length encoded string.
- Compare original and encoded lengths.
- Keep the original when compression does not save space.

03 ASSUMPTIONS

- The input is a non-empty string for this example.
- Characters are case-sensitive.
- The encoded format is character followed by count, including count 1.
- The program selects the shorter representation.

04 ALGORITHM

1. Start with a sample string.
2. If the string is empty, handle it separately.
3. Track the current character and its count.
4. When the character changes, append the previous character and count.
5. Append the final character run.
6. Compare encoded and original lengths.
7. Display both and the chosen representation.
8. Stop.

05 FLOWCHART (TEXT FLOW)

START → Read text → Empty? → Yes: handle empty text → No: count consecutive character runs → Build encoded text → Is encoded shorter? → Choose shorter form → Display lengths and result → END

06 PSEUDOCODE

```
BEGIN
  text ← aaabbccccd
  encoded ← ''; count ← 1
  FOR i from 1 to length(text)-1
    IF text[i] = text[i-1] THEN count++
    ELSE append text[i-1] and count; count ← 1
  END FOR
  append last character and count
  IF length(encoded) < length(text) THEN chosen ← encoded
  ELSE chosen ← text
  PRINT text, encoded, chosen
END
```

07 PYTHON SOURCE CODE

```
text = "aaabbccccd"

if text == "":
    encoded = ""
else:
    encoded = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            encoded += text[i - 1] + str(count)
            count = 1

    encoded += text[-1] + str(count)

if len(encoded) < len(text):
    chosen = encoded
else:
    chosen = text

print("Original text:", text)
print("Encoded text:", encoded)
print("Original length:", len(text))
print("Encoded length:", len(encoded))
print("Chosen representation:", chosen)
```

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
aaabbccccd	a3b2c4d1	Encoded form is shorter
abcd	a1b1c1d1	Original form is kept
xxxx	x4	Encoded form is shorter

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

TEXT COMPRESSION

```
Original text: aaabbccccd
Encoded text: a3b2c4d1
Original length: 10
Encoded length: 8
Chosen representation: a3b2c4d1
```

10 CONCLUSION

The program demonstrates string indexing, loops, counters, and a practical comparison of original and encoded lengths.

CASE STUDY 09

Student Marks Matrix Analysis

ASSIGNMENT CASE NO. 42	LEVEL Beginner

01 PROBLEM STATEMENT

Analyse a matrix of student marks across subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance.

Problem interpretation

Marks are stored in a nested list where each row represents a student and each column represents a subject. The program calculates student totals and subject averages, then identifies the top student and subjects with low averages.

02 OBJECTIVES

- Represent marks using a nested list.
- Calculate each student's total and average.
- Calculate average marks for each subject.
- Identify the top performer and subjects with averages below a threshold.

03 ASSUMPTIONS

- Rows represent students and columns represent subjects in the same order.
- Marks are between 0 and 100.
- The subject-average warning threshold is 50.
- The example contains at least one student and one subject.

04 ALGORITHM

1. Start.
2. Store student names, subject names, and the marks matrix.
3. For each student, calculate the row total and average.
4. For each subject, sum the column and divide by student count.
5. Find the student with the highest total.
6. Flag subjects with average below 50.
7. Display all statistics.
8. Stop.

05 FLOWCHART (TEXT FLOW)

START → Store names and marks matrix → Calculate row totals/averages → Calculate column averages → Find highest total → Check low subject averages → Print analysis → END

06 PSEUDOCODE

```
BEGIN
  students ← [Asha, Ravi, Noor]
  subjects ← [Python, Maths, English]
  marks ← [[80, 75, 90], [65, 85, 70], [45, 55, 60]]
  FOR each student row: calculate total and average
  FOR each subject column: calculate average
  identify student with maximum total
  flag subject averages below 50
  PRINT report
END
```

07 PYTHON SOURCE CODE

```
students = ["Asha", "Ravi", "Noor"]
subjects = ["Python", "Maths", "English"]
marks = [
    [80, 75, 90],
    [65, 85, 70],
    [45, 55, 60]
]
low_average_limit = 50

print("STUDENT TOTALS AND AVERAGES")
totals = []
for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)
    totals.append(total)
    print(f"{students[i]}: total={total}, average={average:.2f}")

print("\nSUBJECT AVERAGES")
subject_averages = []
for j in range(len(subjects)):
    column_total = 0
    for i in range(len(students)):
        column_total += marks[i][j]
    average = column_total / len(students)
    subject_averages.append(average)
    print(f"{subjects[j]}: {average:.2f}")

top_index = totals.index(max(totals))
print(f"Top performer: {students[top_index]} ({totals[top_index]} marks)")
low_subjects = []
for j in range(len(subjects)):
    if subject_averages[j] < low_average_limit:
        low_subjects.append(subjects[j])
print("Subjects below average threshold:", low_subjects if low_subjects else "None")
```

08 TEST CASES

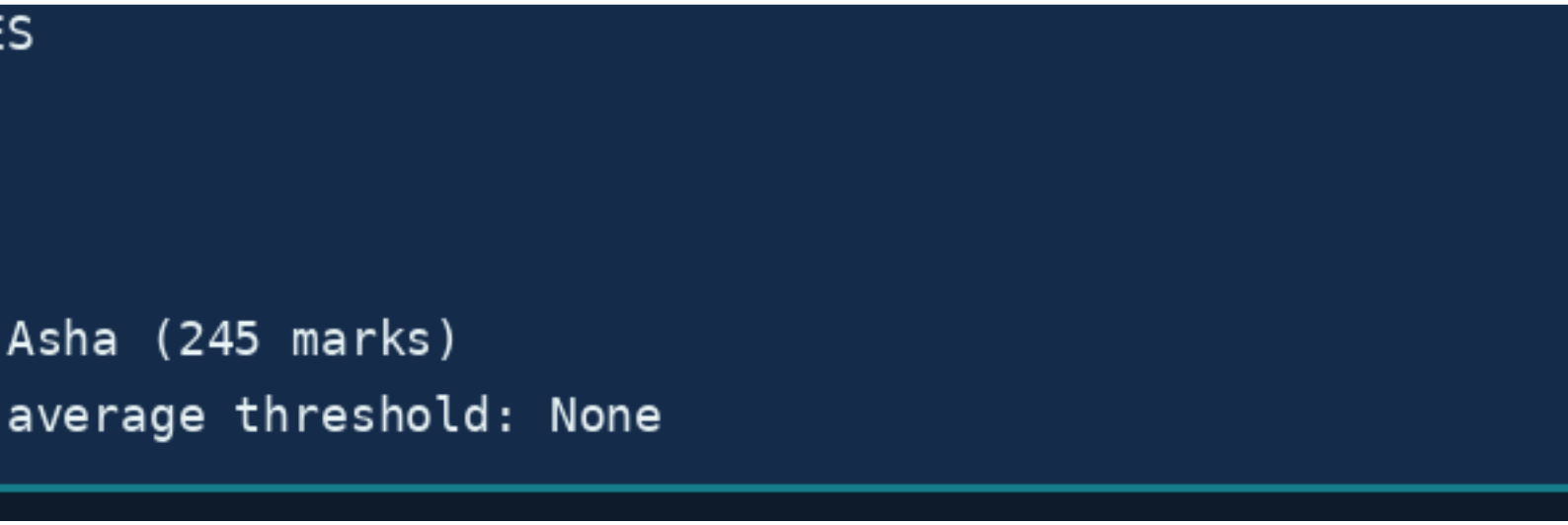
INPUT / SCENARIO	CHECK	EXPECTED RESULT
Asha: 80,75,90	Total 245	Top performer
Python marks 80,65,45	Average 63.33	Not flagged
Subject average 45	Below 50	Flagged

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

```
STUDENT TOTALS AND AVERAGES
Asha: total=245, average=81.67
Ravi: total=220, average=73.33
Noor: total=160, average=53.33

SUBJECT AVERAGES
Python: 63.33
Maths: 71.67
English: 73.33
Top performer: Asha (245 marks)
Subjects below average threshold: None
```



10 CONCLUSION

The program uses nested lists and nested loops to calculate row-wise and column-wise statistics and identify the top performer.

CASE STUDY 10

Food Waste Monitoring System

ASSIGNMENT CASE NO. 53	LEVEL Beginner

01 PROBLEM STATEMENT

Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

Problem interpretation

A canteen records meals prepared and meals served each day. The program calculates leftovers and wastage percentage, then flags days where the wastage percentage exceeds the permitted limit.

02 OBJECTIVES

- Store daily meal counts in lists.
- Calculate leftover meals.
- Calculate wastage percentage.
- Flag days exceeding the acceptable wastage limit.

03 ASSUMPTIONS

- Meals prepared must be greater than zero.
- Meals served cannot be negative or exceed meals prepared.
- Wastage percentage = leftovers / prepared × 100.
- The acceptable wastage limit is 10%.

04 ALGORITHM

1. Start.
2. Store days, meals prepared, and meals served.
3. For each day validate the counts.
4. Calculate leftovers and wastage percentage.
5. Compare the percentage with the 10% limit.
6. Display daily results and flag high-waste days.
7. Stop.

05 FLOWCHART (TEXT FLOW)

START → Store daily data → Validate counts → Calculate leftovers and waste % → Above 10%? → Flag if yes → Display daily report → END

06 PSEUDOCODE

```
BEGIN
  days ← [Mon, Tue, Wed, Thu]
  prepared ← [100, 120, 90, 110]
  served ← [92, 100, 85, 88]
  FOR each day i
    IF prepared[i] <= 0 OR served[i] < 0 OR served[i] > prepared[i] THEN report invalid
    ELSE leftovers ← prepared[i] - served[i]
      waste_percent ← leftovers / prepared[i] * 100
      IF waste_percent > 10 THEN flag day
    END IF
  END FOR
END
```

07 PYTHON SOURCE CODE

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday"]
prepared = [100, 120, 90, 110]
served = [92, 100, 85, 88]
waste_limit = 10
high_waste_days = []

print("FOOD WASTE REPORT")
for i in range(len(days)):
    if prepared[i] <= 0 or served[i] < 0 or served[i] > prepared[i]:
        print(days[i], "- invalid meal counts")
    else:
        leftovers = prepared[i] - served[i]
        waste_percent = leftovers / prepared[i] * 100
        status = "HIGH WASTE" if waste_percent > waste_limit else "Within limit"
        print(f"{days[i]}: prepared={prepared[i]}, served={served[i]}, "
              f"leftovers={leftovers}, waste={waste_percent:.2f}% - {status}")
        if waste_percent > waste_limit:
            high_waste_days.append(days[i])

print("Days exceeding", waste_limit, "%:", high_waste_days if high_waste_days else "None")
```

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
Prepared 100, served 92	8% wastage	Within limit
Prepared 120, served 100	16.67% wastage	Flagged
Prepared 100, served 120	Served exceeds prepared	Invalid meal counts

09 SAMPLE OUTPUT PREVIEW

SAMPLE OUTPUT

FOOD WASTE REPORT

```
Monday: prepared=100, served=92, leftovers=8, waste=8.00% - Within limi
Tuesday: prepared=120, served=100, leftovers=20, waste=16.67% - HIGH WA
Wednesday: prepared=90, served=85, leftovers=5, waste=5.56% - Within li
Thursday: prepared=110, served=88, leftovers=22, waste=20.00% - HIGH WA
Days exceeding 10 %: ['Tuesday', 'Thursday']
```



10 CONCLUSION

The program calculates leftovers and waste percentages, applies a threshold, and helps a canteen identify days that need better meal planning.